

3.1 - TRANSFORMERS

CHAPTER INDEX

- ☐ [LANGUAGE MODELS](#)
 - [TRANSFORMERS AND LLMs](#)
- ☐ [THE TRANSFORMER MODEL](#)
 - [ENCODER](#)
 - [DECODER](#)
 - [RECAP - TRAINING AND INFERENCE](#)
- ☐ [ATTENTION](#)
 - [QUERY - KEY - VALUE IDEA](#)
 - [SELF ATTENTION VS CROSS ATTENTION](#)
 - [MULTI HEAD ATTENTION](#)
 - [MASKED ATTENTION](#)
- ☐ [FURTHER DETAILS](#)
 - [TOKENISATION AND EMBEDDING](#)
 - [POSITIONAL ENCODING](#)
 - [ADD-AND-NORM AND LINEAR LAYERS](#)
 - [ENCODER/DECODER-ONLY ARCHITECTURES](#)
- ☐ [TRANSFORMER EXAMPLES](#)
 - [GPT-OSS](#)
 - [ViT - VISION TRANSFORMER](#)

LANGUAGE MODELS

Before we begin to discuss **transformer models**, it is worth taking a brief detour and look at **language models**.

LANGUAGE MODEL

A **language model** models the **probability distribution** over a **sequence of discrete tokens**:

$$\mathbf{x} = (x_1, \dots, x_T)$$

Where each **token** x_t takes a **value** from a **vocabulary** $\mathcal{V}$.

These **tokens** can be **words**, single **characters** or **raw bytes** depending on the model. Furthermore, the **last token** in a sequence needs to **signals its end**: we thus denote it with the `<EOS>` symbol.

Now, one may wonder what "*modelling the probability distribution of a sequence*" means.

Formally, we define this probability as follows:

$$\begin{aligned} p(\mathbf{x}) &= p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \\ &= p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \dots \end{aligned}$$

Notice how we are computing the **probability** that the **first token** is x_1 , then the probability that the **second token** is x_2 given that the **first was** x_1 , and so on.

To understand this better, consider the following example:

$$p(\text{The dog ran away } \langle \text{EOS} \rangle) = p(\text{The})p(\text{Dog}|\text{The})p(\text{ran}|\text{The dog})$$

$$p(\text{away}|\text{The dog ran})p(\langle \text{EOS} \rangle|\text{The dog ran away})$$

💡 OBSERVATION

Intuitively, we are **predicting the next token** in a sequence, **given** the fact that we already have the **previous tokens**

TRANSFORMERS AND LLMs

The nature of the problem seems to suggest that we should use some **sequence-based model architecture**, such as **LSTM** (see > [3.0 - SEQUENCE AND GRAPH-BASED NNs > LONG SHORT-TERM MEMORY](#)); but we should also notice that **language modelling** comes with another **important detail**: a word's **meaning** may **change depending** on **other words in the sentence**.

Consider the following two examples:

☰ EXAMPLES

1. She poured water from the pitcher to the cup until **it** was **full**.
2. She poured water from the pitcher to the cup until **it** was **empty**.

Notice that in the first sentence, "*it*" refers to the *cup*; but in the second sentence *it* refers to the *pitcher*: this change simply depends on the fact that *full* was substituted by *empty*.

💡 OBSERVATION

Meaning is a result of relationship between words.

This problem can be tackled with what is referred to as the "**attention mechanism**" (we will discuss exactly how it works in the paragraphs below. See > [ATTENTION](#)),

which was indeed applied to LSTM and other RNN architectures to **improve the quality** of their **predictions**.

💡 MOVING FORWARD

The **attention mechanism** improves the quality of a language model, but **LSTMs** are inherently **slow** due to the fact that training requires processing **one token at a time**.

*Could we design a neural network architecture that makes **use** of this **attention mechanism**, but that also allows for **better parallelisation**?*

Indeed we can, and such architecture is exactly the **transformer**, which **enabled** the development of **modern LLMs** (*large language models*).

The main idea behind these models was to **process the entire sequence at once** (hence allowing parallelisation that wasn't possible with LSTMs) and, for **each token** in the sequence, **compute its "attention"** with respect to **all the other tokens**.

To understand just how **important** the attention mechanism is, consider that **Google** researchers almost used the term as the **name for their 2017 model**.

Even though they opted for a different name (*transformer*), the **original paper** ***Attention is all you need*** (probably one of the most well-known papers in the field) effectively reflects this importance.

✍️ CURIOSITY: ORIGIN OF THE NAME "TRANSFORMER"

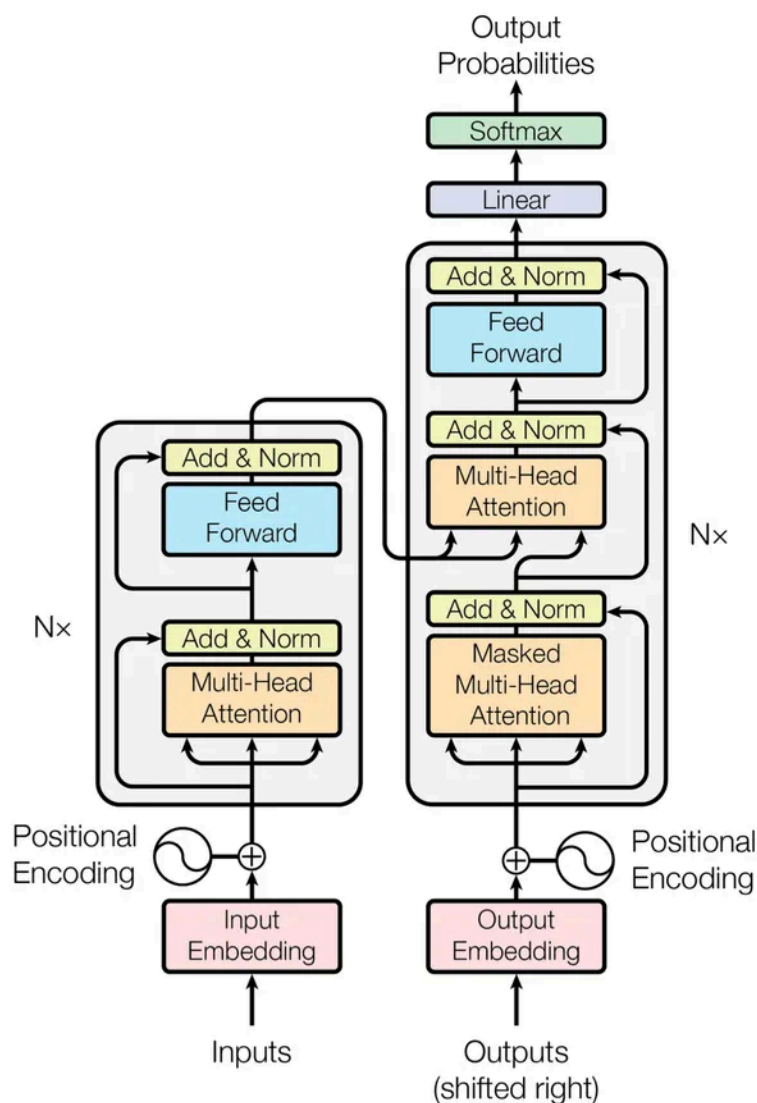
Vaswani, one of the **researchers**, has said:

"I argued we were transforming representations, but that was just playing semantics."

THE TRANSFORMER MODEL

Now that we've given some information about the historic background behind **transformers** and their importance for LLMs, we can finally start diving into **how they work** in more detail.

Particularly, let us begin by taking a look at the **transformer architecture** (shown in the figure below) to build a *general understanding* of the **flow of information through the network**.



For a start, we immediately notice that the *core* of these networks comprises **two main blocks**: an **encoder** (left) and a **decoder** (right).

To understand why we need both, imagine we're dealing with a **sequence to sequence problem**, like **translation**.

Particularly, suppose our **input sequence** is [How, are, you?, <EOS>] and we want to **output** the **italian translation** [Come, stai?, <EOS>] .

Ideally, we want to train an **encoder** to produce an **information-rich latent representation** of the **input**, so that the **decoder** can **leverage** it to produce a **relevant/accurate prediction**.

This process may seem familiar, and indeed the idea behind the **transformer encoder** shares some **similarities** with **autoencoders** (which we discussed in > [2.0 - CONVOLUTIONAL NEURAL NETWORKS > AUTO ENCODER](#)).

To better understand the **role** of the **encoder**, let us go back to the **language translation** example proposed above, and let us consider the following observation: **input** and **output** are **naturally represented** in **different languages**.

At this point, the **role** of the **encoder-decoder architecture** should become clearer:

💡 ENCODER - DECODER: INTUITION

The **encoder** produces a **latent space representation** that **does not depend** on the **particular languages** of the input and the output.

This allows the **network** to **learn concepts** in a **space** that **encapsulates their inherent semantics**, without being constrained by one language in particular.

Then, clearly, we also need a **decoder** to **"convert"** the outputs back to **natural language**.

As for our comparison with autoencoders seen in the CNN chapter, let us clarify the following detail:

✍ NOTE: AUTOENCODERS AND TRANSFORMER ENCODERS

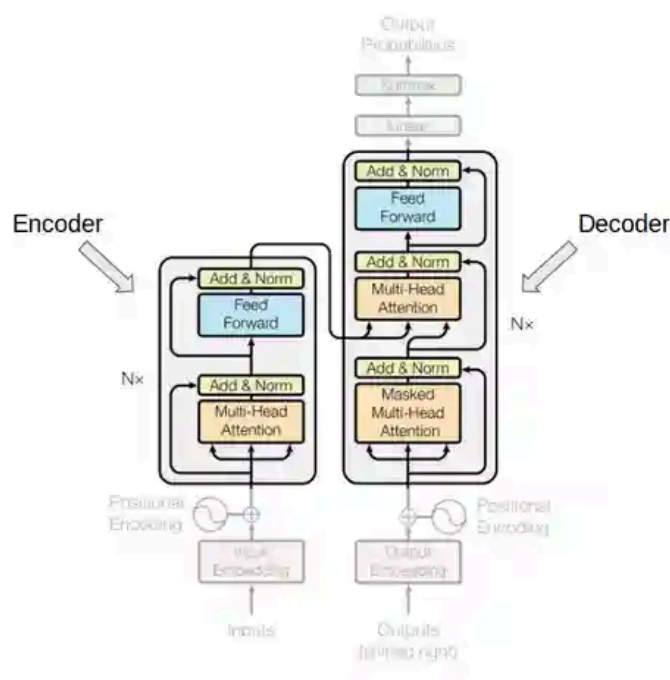
While **autoencoders** typically **compress inputs** to **reduce their dimensionality**, transformer **encoders** learn a **new representation** for the inputs that **retains positional and contextual information**.

Still, in **both cases**, the **new representation** serves to allow the **network** to **learn complex features** more easily.

ENCODER

Now that we have a **general picture** of the transformer architecture, we can discuss in more detail how the **encoder** and **decoder** blocks actually work.

Let us start with the **encoder**.



The **encoder** takes the **embedded input sequence** (that is, each **token** is **embedded in a vector**). Furthermore, the sequence is also injected with **positional information** thanks to **positional encoding**) and **processes** it through a **stack of layers** (notice the $N \times$: we can stack as many blocks as needed).

✎ ENCODER BLOCKS

Each block consists of:

1. An **attention layer**, which allows **every token** to **look at** all the **others** to build a **context-aware representation**.
2. A **feed-forward network**, which **refines** the information-rich representation produced by the attention layer.

Notice that we're only giving a brief description of the role of the attention mechanism, which we will discuss more thoroughly in > [ATTENTION](#).

DECODER

Now, understanding the **decoder** is a bit more **complex**, particularly because we need to distinguish the *input-feeding process* for **inference** from that used during **training**.

It is probably easier if we go over the **training** process first; and again in the context of *language translation* that we used in the example above.

To this end, we should start by addressing what we mean with the "**shifted right**" note on the **decoder input**.

The output that is "*shifted right*" is the **target output** that is associated with a particular input. Thus, during the **training** phase, we take the **target outputs**, **right-shift** them and then **feed them** to the network.

In our case the target output was [Come, stai?, <EOS>] ; shifting it to the right means **dropping the <EOS> token** and **adding an additional special token to the front**, thus obtaining [<SOS>, Come, stai?] (this special token is **Start Of Sentence**).

To understand the reason behind this shift, consider the two next to each other:

POS. IN SEQ.	DECODER INPUT	DECODER TARGET
1	<SOS>	Come
2	Come	stai?
3	stai?	<EOS>

💡 OBSERVATION: INPUT-TARGET ALIGNMENT

Notice that <SOS> is aligned with Come , Come with stai? , and stai? with <EOS> .

That is, **after the right shift**, each **input token** is correctly **aligned** to its **target prediction** in the **output** sequence.

When the input has been fed to the network, the **decoder** computes a **masked attention**, then runs the output through **another attention layer**, which also receives the **encoder output**.

Because this layer computes attention between **both** encoder and decoder representations, we also call it **cross attention**.

Then, the output is **refined** by a **feed-forward layer** and a **normalisation layer**.

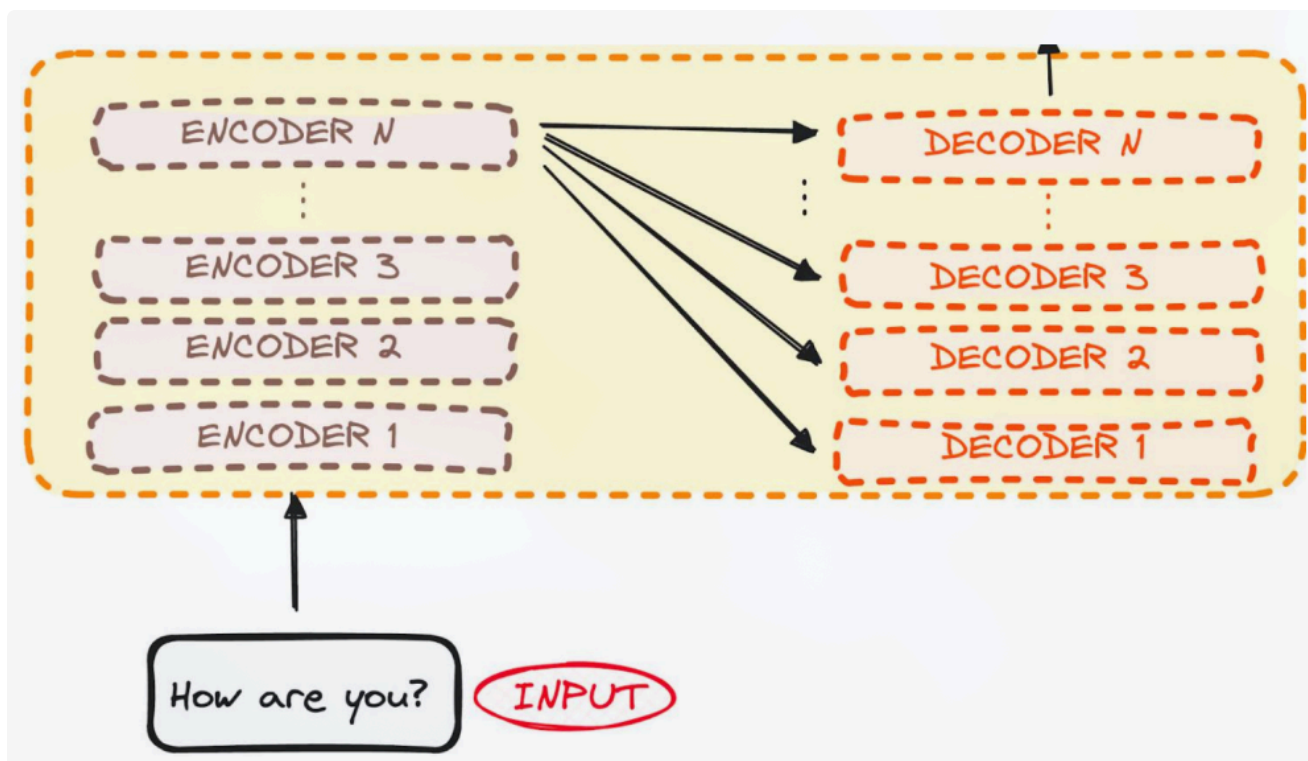
Finally, after N **decoder blocks** (these can be stacked together just like the encoder blocks), a **linear layer** equipped with the **softmax activation function** generates the **output prediction**.

Before looking at how the decoder operates during inference, let us clarify a few details:

DECODER - MASKED ATTENTION AND CROSS ATTENTION

- **MASKED ATTENTION:** Recall that, unlike RNNs, the **transformer training** process looks at the **whole sequence at once**.
Masked attention thus serves as a way to **prevent the network** from **looking at future tokens**. That is, by **applying a mask** we can **ensure** that **attention** is computed with respect to **past tokens** only: the model must learn to predict new tokens from the previous ones; looking at future tokens would be "cheating" (see the > [MASKED ATTENTION](#) paragraph below).
- **CROSS ATTENTION:** Computing **attention** between the **encoder outputs** and the **decoder's intermediate representations** allows the network to **learn which parts** of the **input** sequence are **relevant** to compute the **output** (this mechanism is better explained in the > [QUERY - KEY - VALUE IDEA](#) and > [SELF ATTENTION VS CROSS ATTENTION](#) paragraphs below).

Furthermore, notice that **every encoder** block receives the **output of the last encoder**, as shown below:



Now we can finally move on to **inference**, where the information flow is slightly different.

Particularly, once the **encoder** has produced its **latent space representation**, we need to **query it** (again, see > [QUERY - KEY - VALUE IDEA](#) below for details) to **retrieve information** that we need to **produce the next token**.

The **inference** process thus looks as follows:

DECODER - INFERENCE

1. We **start by feeding <SOS>** to the **decoder**.
2. The **cross-attention mechanism** retrieves **information** from the **encoder**.
3. This information is used to generate the **next token**, which is **appended to <SOS>**.
4. We **repeat the steps**, but we **feed the whole sequence generated so far** instead of just <SOS>.
5. We **keep generating new tokens until <EOS>** is outputted by the decoder.

RECAP - TRAINING AND INFERENCE

At this point, it might be useful to recap a few **key differences** between the training and inference processes:

INFERENCE VS TRAINING

TRAINING

1. **Sequence processing:** the **encoder** receives the **entire source sequence**, the **decoder** receives the **right-shifted target sequence**.
2. **Masking:** necessary to **prevent** the model from **looking at future tokens**.
3. **Output:** the decoder outputs a **prediction for every position simultaneously** (and thus loss is also computed across all positions at once).

INFERENCE

1. **Sequence processing:** the **encoder** receives the **entire source sequence**, while the **decoder** operates in a **loop** to **autoregressively generate the output**.
2. **Masking:** not needed, because the "**future tokens**" **haven't been generated yet**.
3. **Output:** the decoder outputs the **final sequence** by adding **one token at a time** (the most likely one given the sequence produced so far) **until <EOS>**.

Furthermore, recall that at the beginning of the chapter (see > [TRANSFORMERS AND LLMs](#)) we presented transformers as an **alternative to LSTMs** that allows for **better parallelisation**.

Now that we know how the **encoder** and **decoder work**, we can clarify this detail:

LSTM VS TRANSFORMER: PARALLELISATION

Both models produce their **output one token at a time**, so where is the advantage of transformers?

The answer is in the **training phase**: while LSTMs need to *process one token at a time, even during training*; **transformers** can be trained on a **whole sequence at once**.

This makes Transformers **much faster to train** compared to RNNs, enabling the massive scale of modern LLMs.

Though we should also notice that this comes at a cost: **memory required** for **transformer attention** scales **quadratically with sequence length T** ($O(T^2)$, see > [ATTENTION](#) paragraph below), whereas LSTM memory scaled linearly ($O(T)$).

ATTENTION

So far, we have gone over the general **architecture** of the **transformer model**, and we have built a basic understanding of the **attention mechanism** as a way to **capture semantic relationships inside sequences**, but without properly discussing **how**.

Because this idea is so important for a complete understanding of transformers, it is worth dedicating a few paragraphs entirely to this concept.

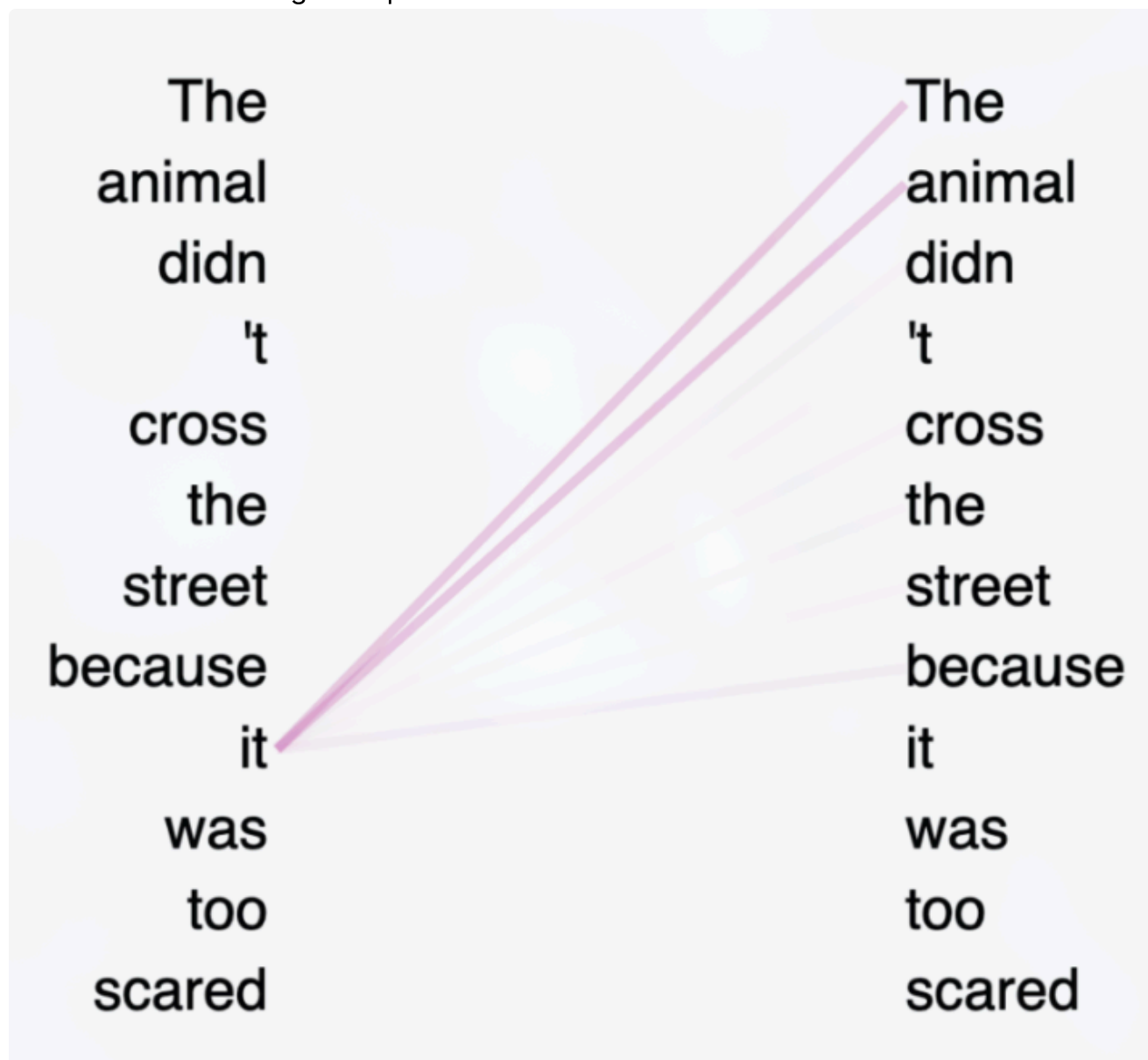
For a start, let's recall what we want to achieve with the attention mechanism:

💡 (SELF) ATTENTION MECHANISM

We want **each token** in the **sequence** to *"understand"* the relevance of all **other tokens** to itself.

That is, we want each token to **look at all the others** and **decide which** ones are **relevant** and which ones **are not**.

Consider the following example:



We can think of **color intensity** as the **strength** of the **relationship** between words.

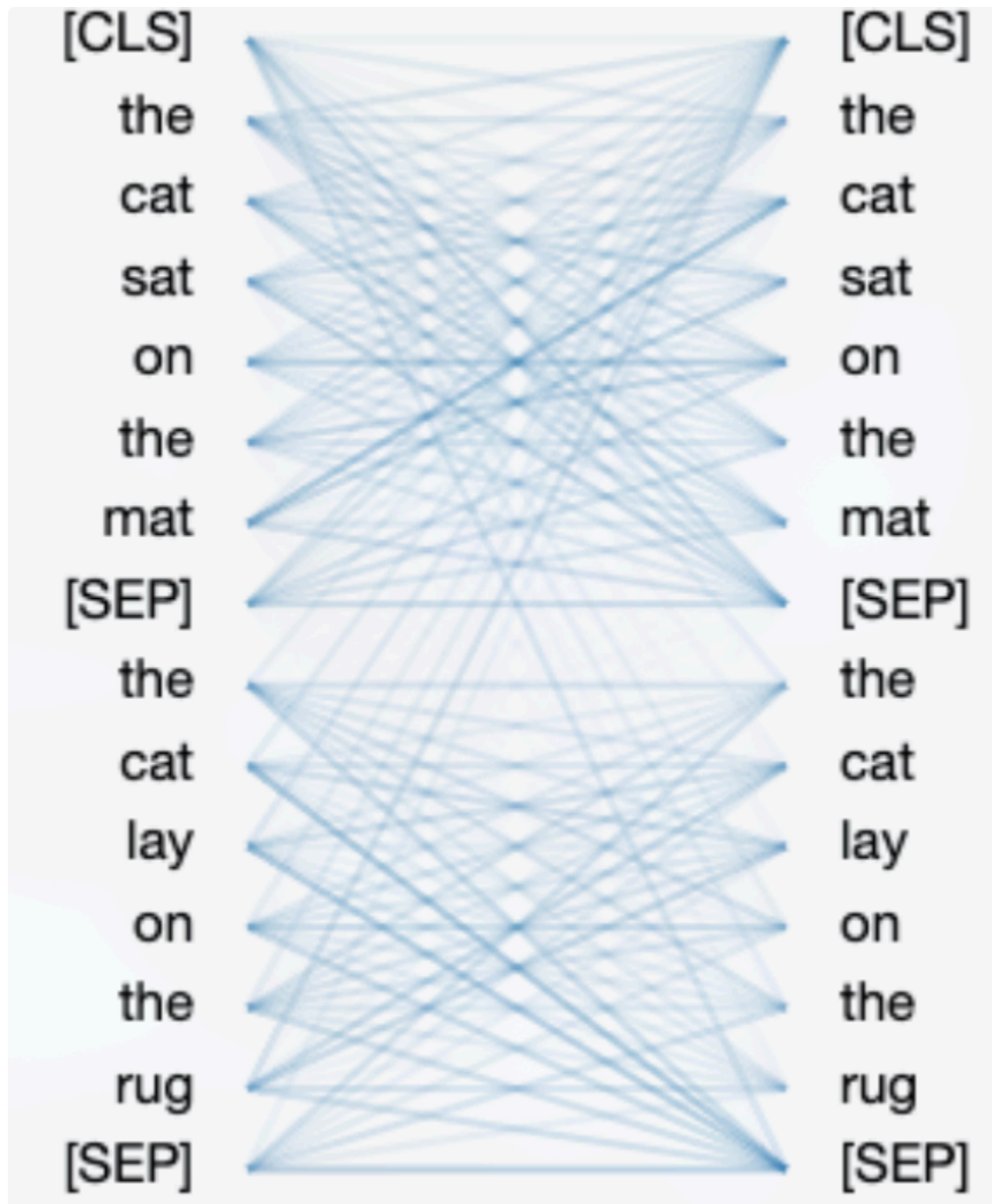
✍ NOTE

We can see that the **word "it"** has:

- **strong connections** with the words "*the*" and "*animal*": we will associate a **high attention score** to these relationships.
- a **weak connection** with the word "*because*": this will be associated with a **low attention score**.
- **(almost) no connections** with the **other words**: we thus want to associate these with **even lower attention scores**.

The figure showed the attention that "it" pays to other tokens.

Clearly, the same can be done for **all other tokens**, resulting in something like:



What we presented here is the idea behind **self-attention**.

Transformers also implement what we call **cross-attention**: though the core idea behind it remains unchanged, there are some differences that we discuss in > [SELF ATTENTION VS CROSS ATTENTION](#).

With this in mind, we will go over the **main idea behind attention**, move on to **how** it is **implemented** in the **transformer model**, and, finally, we will discuss some **further details** about **multi head attention** and **masked attention**.

QUERY - KEY - VALUE IDEA

In the [DECODER](#) paragraph we have hinted at the fact that, ideally, we want the **encoder** to produce some kind of **information-rich representation** which then gets **queried** by the **decoder** to **generate tokens**.

The **mixing between encoder and decoder representations** is the idea behind **cross-attention** (again, see > [SELF ATTENTION VS CROSS ATTENTION](#)), but the **concept of "querying for information"** is the **core intuition** behind the **whole implementation of the attention mechanism in transformers**.

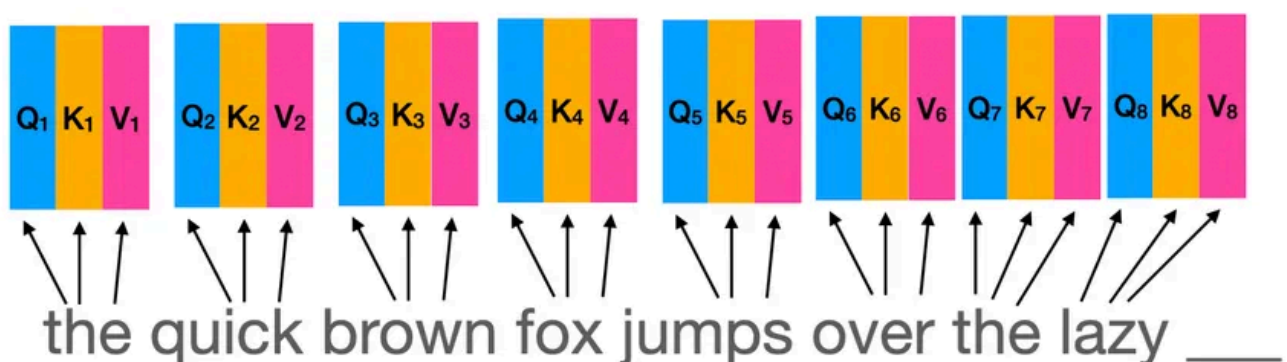
For this reason, we begin by discussing this idea first:

GENERAL IDEA

We want to **submit a query Q** for **each token** in the sequence, **match them against a set of keys K** , and **retrieve values V** associated with **keys that align with the query**.

To formalise this, we first **associate each token** with **three vectors**:

1. **QUERY q** Represents the token's "question" about **what information** it is **looking for** from the **rest of the sequence**.
2. **KEY k** Acts as a **label** (descriptor) that tells the other tokens **what kind of information** is associated with this particular token.
3. **VALUE v** Is the **actual information** associated with the token.



We can think of this mechanism as something analogous to **information retrieval in search engines**.

Indeed, the idea is very much similar, with the only difference given by the fact that we use a **more "abstract" and "smooth" notion of *retrieving an object***: we **retrieve information** if the **associated key is *similar* to our query**.

At this point, the following intuition should come up naturally:

💡 ATTENTION AS SIMILARITY

If the **similarity between keys and queries** acts as a way to **select relevant information**, and this was exactly the idea we wanted to implement with the attention mechanism; then **attention must be a measure of this similarity**.

With this in mind, let us take a look at how the attention mechanism works:

📋 ATTENTION MECHANISM

1. **Compute similarity** between a given **query $\mathbf{q}_i$** and **every key $\mathbf{k}_j$** to produce a score of the **relevance of each token j** to token i .
2. **Normalise** these scores with the **softmax function** to turn them into a **probability distribution**.
3. **Aggregate the values** by **weighting them** with the **normalised scores**.

At the end, the result is a **new representation** for token i that **mixes information** from the **most relevant tokens** in the sequence.

Particularly, we use the **dot product** to determine the **similarity between queries and keys**.

Thus, given a **single query vector $\mathbf{q}_i$** , we compute its **attention to all other tokens in the sequence** as follows:

📖 ATTENTION (SINGLE QUERY)

$$\begin{aligned}\text{Attention}(\mathbf{q}_i, \mathbf{K}, \mathbf{V}) &= \sum_j \text{softmax} \left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \right) \mathbf{v}_j \\ &= \text{softmax} \left(\frac{\mathbf{q}_i \mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V} \in \mathbb{R}^{d_v}\end{aligned}$$

Where $\mathbf{k}_j$ and $\mathbf{v}_j$ are the **key and value** associated with the j -th **token**, and the **index j runs through the entire sequence**.

Notice that the second equation was simply obtained by **combining** all **key** and **value vectors** in **matrices** (**K** and **V**) to introduce a *more compact notation*; and that we *scale* by a *factor* of $\sqrt{d_k}$ where d_k is the *dimension* of the *key* and *query* vectors.

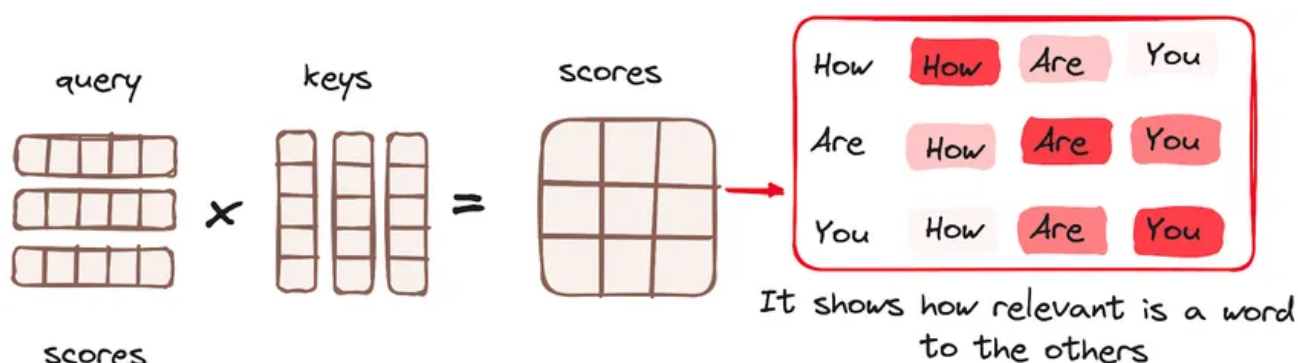
The above definition gives us the **attention** for a **single query vector** q_i .

If **all queries are available**, we can **group** them in a matrix **Q** just like we did with keys and values:

🔗 ATTENTION (ALL QUERIES)

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

This way, the result of $\mathbf{Q}\mathbf{K}^\top$ is what we call **attention matrix**, whose entries a_{ij} are the **strengths** with which **token i** attends to token j .



🔗 NOTE: WE NEED A MASK

Because we **do not want** a **token i** to **look at future tokens $j > i$** , the training phase requires the use of a **mask** (as already briefly introduced in > [DECODER](#)).

This introduces the need for a **masked attention layer** (see > [MASKED ATTENTION](#) paragraph below) in the **decoder blocks**.

SELF ATTENTION VS CROSS ATTENTION

In the previous paragraph we went over the **general idea** behind the **attention mechanism** and **how it is computed**, but we **haven't specified** how **queries, keys and values** are **obtained**.

🔗 SELF-ATTENTION AND CROSS-ATTENTION

Recall (see figure at the start of > [THE TRANSFORMER MODEL](#)) that there are **two kinds of attention layers**.

1. **Self-attention** layers, that compute **attention** entirely within **either** the **encoder** *or* the **decoder** (hence, the "*self*" prefix).
2. **Cross-attention** layers, that compute **attention** on **encoder outputs** *and* **decoder intermediate representations** (hence, "*cross*").

The fundamental difference between them lies in **where Q, K, and V come from**, which also determines their **dimensions**.

To discuss this, let us define some variables first:

- T : **sequence length**.
- d_{model} : dimension of the **input embeddings** (512 in the original paper).
- d_k : dimension of the **query** and **key vectors**.
- d_v : dimension of the **value vectors**.

Let us begin with the **self-attention layers** and an **input sequence** $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$; notice that $\mathbf{X}$ can be the **encoder input**, a **target sequence** for the **decoder** during **training**, or the **sequence generated so far** by the **decoder** during **inference**.

To **compute queries, keys and values** we simply **apply weight matrices** to this sequence; we thus have:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q \in \mathbb{R}^{T \times d_k}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^K \in \mathbb{R}^{T \times d_k}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^V \in \mathbb{R}^{T \times d_v}$$

Notice that $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $\mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

The **weight matrices** map each component of the embedding to **vectors** that represent the **query, key, and value** associated with that **particular representation**. By **repeating this** for each token, we can **map the whole sequence** to the **target (Q, K, V) space**: this means that the **network learns the mappings** between **embeddings** and **queries, keys and values**.

🔗 OBSERVATION: LEARNING AN EFFICIENT MAPPING

Because the **values** are **retrieved** based on the **similarity between queries and keys**, the **attention** mechanism essentially **forces the network to learn** this mapping in such a way that **encapsulates relationships** between **concepts** (that is, **similar concepts** will be associated with **keys** in **similar directions**).

Now let us look at the **dimension** of the **attention matrix**:

$$\mathbf{A} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{T \times d_k} \cdot \mathbb{R}^{d_k \times T} = \mathbb{R}^{T \times T}$$

 NOTE: SELF-ATTENTION IS A SQUARE MATRIX

Entries a_{ij} are the **attention values** of token i to token j .

Equivalently, the **i-th row** is the **attention vector** associated with the **i-th token**.

Note that this matrix is **not symmetric**: we may have $a_{ij} \neq a_{ji}$.

Finally, because we defined T as the **length** of the **sequence** that is **fed** to the **self-attention layer**, we should clarify the following detail:

 NOTE: MATRIX SHAPE IN ENCODER AND DECODER BLOCKS

- **ENCODER:**

The matrix has **simply** shape $T \times T$, where T is the **input sequence length**.

- **DECODER:**

During **training** (where the entire sequence is fed at once), the attention matrix has shape $T_{\text{target}} \times T_{\text{target}}$, where T_{target} is the **target output sequence length**.

During **inference** (where the sequence is **generated one token at a time**), the attention matrix **grows in size at each step**: $A \in \mathbb{R}^{T_t \times T_t}$ with $t = 1, \dots, T_{\text{max}}$ and $T_1 = 1$.

Now, let us do the **same analysis** for the **cross-attention layers**; but let us point out the **main difference** with self-attention first:

 NOTE: MAIN DIFFERENCE

While in **self-attention** we have **queries, keys and values** that all **come from the same source**, when computing **cross-attention**:

- **Queries** come from the **decoder**.
- **Keys** and **values** come from the **encoder**.

Let the **encoder output** be $\mathbf{X}_{\text{enc}} \in \mathbb{R}^{T_{\text{enc}} \times d_{\text{model}}}$ and let the **decoder intermediate representation** be $\mathbf{X}_{\text{dec}} \in \mathbb{R}^{T_{\text{dec}} \times d_{\text{model}}}$.

The **projections** are now:

$$\mathbf{Q} = \mathbf{X}_{\text{dec}} \mathbf{W}^Q \in \mathbb{R}^{T_{\text{dec}} \times d_k}$$

$$\mathbf{K} = \mathbf{X}_{\text{enc}} \mathbf{W}^K \in \mathbb{R}^{T_{\text{enc}} \times d_k}$$

$$\mathbf{V} = \mathbf{X}_{\text{enc}} \mathbf{W}^V \in \mathbb{R}^{T_{\text{enc}} \times d_v}$$

Notice that the **weights matrices** have the **same shapes** as before ($\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$), but **now the queries matrix has a different sequence length than the keys and values.**

As a result, the **cross attention matrix** has the following **dimensions**:

$$C = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{T_{\text{dec}} \times T_{\text{enc}}}$$

Because $T_{\text{dec}} \neq T_{\text{enc}}$, we now have:

NOTE: CROSS-ATTENTION IS A RECTANGULAR MATRIX

The cross-attention matrix is **rectangular** and **row i** tells us how much the i -th **decoder-generated token** attends to **each token** in the **encoder generated representation**.

Just like we did before, we should also clarify a detail about T_{dec} :

NOTE: DECODER SEQUENCE LENGTH

- During the **training phase** we have $T_{\text{dec}} = T_{\text{target}}$, which means we have **target rows**, each with an **attention score** for **each token** in the **representation** generated by the **encoder**.
- During **inference**, $T_{\text{dec}} = T_t$: it **increases** as we **generate new output tokens**. Thus, each time a **new token** is **generated** we **add one row** with the **attention scores** with respect to all the **encoder generated tokens**.

The notes about the **self-attention** and the **cross-attention matrices'** dimensions are **key** to understand the **difference** between the **two kinds of attention**:

SELF-ATTENTION VS CROSS-ATTENTION

- **SELF-ATTENTION** computes **attention scores within a single sequence**. Intuitively, this acts as a way to produce a **representation** that **emphasises** the **relationships** between **elements within the sequence**.
- **CROSS-ATTENTION** computes **attention scores** between the **encoder output** (which, remember, is just **another representation of the input**) and the **decoder output** that is **being generated**. Intuitively, this acts as a way for the **decoder** to **look back** at the **input sequence** and **retrieve** some **relevant source information** that needs to be **carried over** to the **output**, thus making it **more aligned (relevant)** with the **input**.

MULTI HEAD ATTENTION

The authors of the *Attention is all you need* paper found that, **instead** of performing a **single attention function** on d_{model} -dimensional keys and queries, it was **beneficial** to **perform multiple attention functions in parallel**.

In particular, **query, key** and **value** matrices are **mapped to** different, **lower dimensional spaces** of dimensions $d'_k = d_k/h$ and $d'_v = d_v/h$, where h is the **number of times** we want to **repeat the attention mechanism**.

Each **output** is called a **"head"** and is computed as follows:

📖 ATTENTION HEAD

$$\text{Head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V), i = 1, \dots, h$$

Notice that $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d'_k}$ and $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d'_v}$.

Because **only** the **input** is **different**, and the **attention function** remains the **same**, the key takeaway of an attention head is the following:

📖 ATTENTION HEAD

An **attention head** essentially **computes attention** in a **lower dimensional space** compared to the "standard" attention.

Now, because we **stack** h of these **attention heads**, the **final output** of a **multi-head attention layer** can be computed as follows:

📖 MULTI-HEAD ATTENTION

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

Where $\mathbf{W}^O \in \mathbb{R}^{hd'_v \times d_{\text{model}}} = \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$.

🔍 OBSERVATION: MULTI-HEAD ATTENTION RESULT

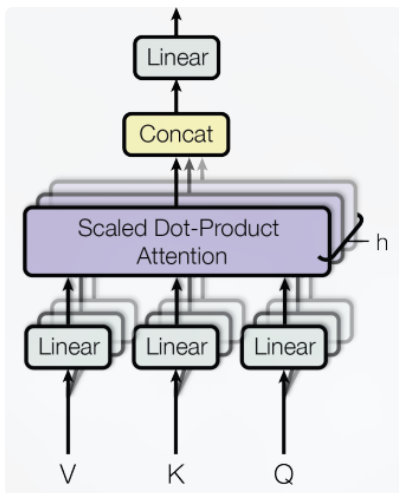
The **attention heads** are **concatenated together** and then **mapped back** to a d_{model} -dimensional **space** that extrapolates the **information carried by each head**.

In their original work, the authors employed $h = 8$ and $d'_k = d'_v = d_{\text{model}}/h = 64$.

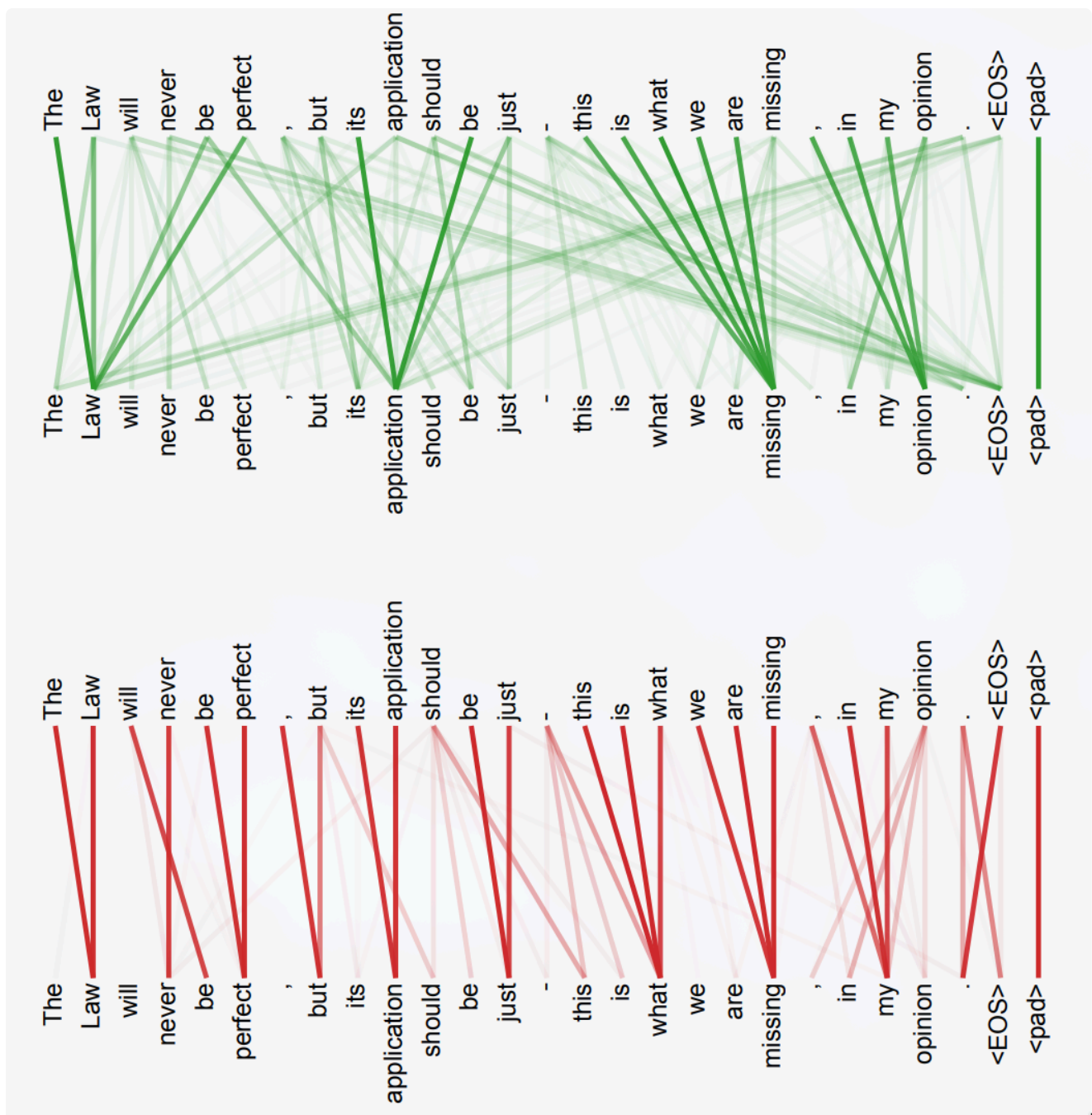
Based on the definition of multi-head attention, we can notice a few important details:

NOTE: DETAILS

1. Because **heads** are **independent of each other**, we can perform their **computation in parallel**.
2. Due to the **reduced dimension of each head**, the **total computational cost** is **similar** to that of **single-head attention** with the original **full dimensionality**.



Finally, it is worth taking a look at the figure below to understand **why multi-head attention** may be **beneficial**:



The image (which is also shown in the original paper) displays **attention values** in **two attention heads** within the **same layer** for the example sentence.

We can clearly see that the two heads have **learned different connections**.

🧐 INTUITION: MULTI-HEAD ATTENTION

Having **multiple attention heads** allows the model to **look at different representation subspaces**, which may **capture different relationships** between tokens, thus **improving both generalisation capabilities and output quality**.

MASKED ATTENTION

In the previous paragraphs we briefly introduced the **need for a mask** during the **training phase** to prevent the **decoder** from **computing attention** with respect to

"future" tokens.

To understand **why this would be a problem** in the first place, let us consider the following example.

☰ TRAINING EXAMPLE

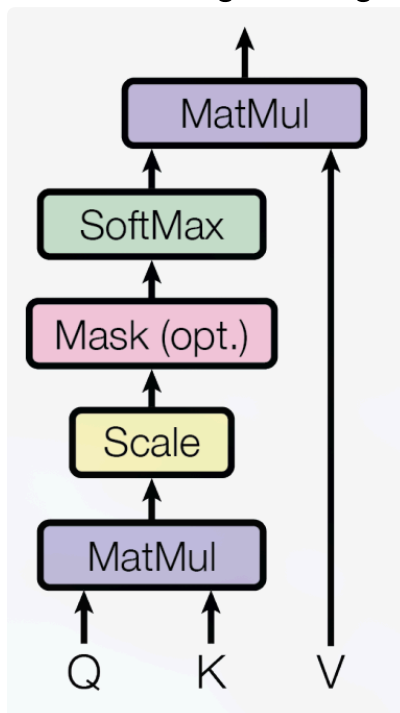
- **TARGET SEQUENCE:** [I, am, fine, <EOS>] .
- **TRAINING INPUT:** [<SOS>, I, am, fine] (*right-shifted target sequence*).

✍ THE PROBLEM

If we were to **generate the sequence during inference**, each time we **add a new token** we would **compute its attention to past tokens only** (e.g. when we add `am`, we would compute its attention to `<SOS>` and `I`. Token `fine` hasn't been generated yet, so `am` does not attend to it).

During **training**, on the other hand, we have the **whole sequence available** and there is **nothing stopping us** from computing the **attention** that `am` pays to `fine`: the network would **"cheat"** the training process by **looking at the future**.

To solve this problem we add a **mask matrix** (or **causal mask**) to the **attention matrix** before running it through softmax:



Mathematically, this mask is defined as follows:

📖 CAUSAL MASK

The **causal mask** is an **upper triangular matrix** of $-\infty$ values of the form:

$$\mathbf{M}_{\text{causal}} = \begin{bmatrix} 0 & -\infty & -\infty & \dots \\ 0 & 0 & -\infty & \dots \\ 0 & 0 & 0 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

💡 OBSERVATION: MASK STRUCTURE

Recall that **each row i** in the **attention matrix** contains the **attention scores** with which **token i** **attends** to the **others**.

Because we want a **token i** to **only attend** to **tokens $j \leq i$** , we set $m_{ij} = -\infty$ for all **positions** where $j > i$.

To **add it** to the **computed attention** before the **softmax function**, we simply compute **masked attention** as follows:

📖 MASKED ATTENTION

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}_{\text{causal}} \right) \mathbf{V}$$

Notice that, after **adding the causal mask**, the **masked positions** take $-\infty$ as their value.

Thus, their **output** after `softmax` is **applied** is **zero** (because $e^t \rightarrow 0$ as $t \rightarrow -\infty$).

Now, one may wonder **how we handle the variable length** in the **training examples**. That is, imagine that **within a batch** we have some sequences of **length 3** and some other of **length 5**. This would **require causal masks of different sizes**.

To overcome this issue, we add special **padding tokens** `<PAD>` to the **end** of the **shorter sequences** so that they **all have the same length**.

Then, since we **do not want** our tokens to **attend to padding tokens**, we also define a **padding mask**:

📖 PADDING MASK

It is a **square matrix** of the form:

$$\mathbf{M}_{\text{pad}} = \begin{bmatrix} 0 & \dots & 0 & -\infty & \dots & -\infty \\ 0 & \dots & 0 & -\infty & \dots & -\infty \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & \dots & 0 & -\infty & \dots & -\infty \end{bmatrix}$$

That is, we have $m_{ij} = -\infty$ where $j \geq \text{original sequence length}$ (equivalently, $m_{ij} = -\infty$ if $S[j] == \text{<PAD>}$, where S is the *training example sequence*).

In practice, we **combine** the **causal mask** and the **padding mask** into a **single mask**:

$$\mathbf{M}_{\text{final}} = \mathbf{M}_{\text{causal}} + \mathbf{M}_{\text{pad}}$$

Which will look like:

$$\mathbf{M}_{\text{final}} = \begin{bmatrix} 0 & -\infty & -\infty & -\infty & \dots & -\infty \\ 0 & 0 & -\infty & -\infty & \dots & -\infty \\ 0 & 0 & 0 & -\infty & \dots & -\infty \\ 0 & 0 & 0 & -\infty & \dots & -\infty \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & -\infty & \dots & -\infty \end{bmatrix}$$

And we **compute attention** by applying the **full mask**:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}_{\text{final}} \right) \mathbf{V}$$

FURTHER DETAILS

We have gone over the **transformer architecture** and presented the **main ideas** behind this model, but there are still some more **details** that are **worth discussing**.

Thus, the following chapters aim at clarifying the details that we have so far omitted.

TOKENISATION AND EMBEDDING

We have been **assuming** that the **input** to the model is **already a sequence of vectors**. In reality, the raw input is **text** (a string of characters/words); but a transformer can only operate on **numbers**.

We thus **need a way** to **convert text into a numerical representation** so that the network can process it.

Particularly, we need to do this in **two steps**:

NOTE: TRANSFORMING TEXT INTO VECTORS

1. **TOKENISATION** - We kept mentioning "**tokens**" as the **individual bits** that we feed to the model; these are **obtained** by **splitting the original sequence** into **smaller units** (e.g. words or characters).

2. **EMBEDDING** - We then need to **map each token** to a dense **vector** that **retains its semantics**.

Let us start by discussing **tokenisation**, which includes a **sequence of steps**:

TOKENISATION PIPELINE

- **NORMALISER** - Applies **Unicode normalisation** (e.g. converting curly quotes to straight quotes), **lowercasing** (optional), **strips accents** and **removes control characters** to **reduce variance in the vocabulary**.
- **PRE-TOKENISER** - Breaks text into **candidate pieces** by **applying basic rules** (e.g. splitting at whitespaces or applying a simple regex).
- **TOKENISATION MODEL** - **Splits** the **pre-tokenised chunks** into **subwords** based on the **learned vocabulary** (e.g. "*unhappiness*" might not be in the vocabulary, but "*un*", "*happy*" and "*ness*" probably are).
- **POST-PROCESSOR** - Adds **special tokens** (e.g. `<SOS>` , `<EOS>` , `<PAD>`) and **maps each token to an integer ID** from the vocabulary.

For example, the sentence "Hello, I'm a TransFormer!" might go through the following steps:

1. "Hello, I'm a transformer!"
2. [Hello],[,],[I'm][a][transformer][!]
3. [Hello],[,],[I]['m][a][transform][er][!]
4. [`<SOS>`][Hello],[,],[I]['m][a][transform][er][!][`<EOS>`] →
[101,7592,1010,1005,1045,1049,1037,10938,2121,999]

At this point, the **integer IDs** produced by the post-processor are the **input** to the **embedding layer**, which simply acts as a **lookup table**.

That is, given a **token ID** `t_ID` we compute its **embedding** as:

```
emb[t_ID] = Vocabulary[t_ID, :]
```

OBSERVATION: VOCABULARY

The **vocabulary** is a **matrix $\mathbf{E}$** of shape:

$$\mathbf{E} \in \mathbb{R}^{d_{\text{vocab}} \times d_{\text{emb}}}$$

Where d_{vocab} is the **number of words** in the **vocabulary**, and d_{emb} is the **embedding dimension**.

Thus, **retrieving the embedding of a token** consists in **reading a row** from the **vocabulary matrix**.

Furthermore, it is important to point out that in **modern transformers** the **embedding matrix** is **learned during training**.

Early NLP models, on the other hand, employed a **static, pre-trained embedding table**. These were obtained with methods such as **Word2Vec**, by **training the algorithm** on a **separate dataset** (text corpus).

STATIC VS LEARNED EMBEDDINGS

Because **static embeddings** are **learnt by an independent algorithm**, they may **not suit a specific application domain** very well.

Contrarily, if we **let our model learn embeddings on its own**, there is a good chance that these will **fit the task better**.

POSITIONAL ENCODING

After the **tokenisation and embedding**, the input sequence receives a **positional encoding addition**, which consists in **adding a set of small constants** to the word embedding vectors before feeding them to the first attention layer.

WHY POSITIONAL ENCODING

Embeddings alone are **not sufficient to retain positional information**.

That is, a **word's embedding** is **always the same**, **independently of its position in the sentence**.

Thus, we would like to **change the representation** of a token so that **it changes depending on *where* it appears in the sentence**.

The authors of the **original paper** decided to employ **sinusoidal positional encoding** to address this issue.

SINUSOIDAL POSITIONAL ENCODING

For **every position t** in the **input sequence** (e.g. $t = 0$ for the first token, $t = 1$ for the second, and so on) we **add the following vector** to the **token embedding**:

$$\mathbf{p}_t = \begin{bmatrix} \sin(\omega_1 t) \\ \cos(\omega_1 t) \\ \sin(\omega_2 t) \\ \cos(\omega_2 t) \\ \vdots \\ \sin(\omega_{d/2} t) \\ \cos(\omega_{d/2} t) \end{bmatrix} \in \mathbb{R}^d$$

Where d is the **embedding dimension**, and the **frequencies** ω_k are given by:

$$\omega_k = \frac{1}{10000^{2k/d}}, k = 1, \dots, d/2$$

The actual **encoder input** is thus given by **adding** the **word embedding vector** and the **positional encoding vector** for **each token** in the **sequence**.

Now, one may wonder **why** we should **use sinusoidal functions** of this kind.

To answer this, let us consider just **one frequency pair** at **position** t :

$$\text{PE}_{(t,2i)} = \sin(\omega_i t)$$

$$\text{PE}_{(t,2i+1)} = \cos(\omega_i t)$$

Now, let us see **what happens** when we **shift the position** t by a **fixed offset** Δt :

$$\text{PE}_{(t+\Delta t, 2i)} = \sin(\omega_i(t + \Delta t)) = \underbrace{\sin(\omega_i t) \cos(\omega_i \Delta t)}_u + \underbrace{\cos(\omega_i t) \sin(\omega_i \Delta t)}_v$$

$$\text{PE}_{(t+\Delta t, 2i+1)} = \cos(\omega_i(t + \Delta t)) = \underbrace{\cos(\omega_i t) \cos(\omega_i \Delta t)}_u - \underbrace{\sin(\omega_i t) \sin(\omega_i \Delta t)}_v$$

Notice that u and v **do not depend on the position** t , but **only** on the **fixed offset**.

We can thus rewrite the equations as follows:

$$\text{PE}_{(t+\Delta t, 2i)} = u \cdot \text{PE}_{(t, 2i)} + v \cdot \text{PE}_{(t, 2i+1)}$$

$$\text{PE}_{(t+\Delta t, 2i+1)} = -v \cdot \text{PE}_{(t, 2i)} + u \cdot \text{PE}_{(t, 2i+1)}$$

Equivalently, we can write:

$$\begin{pmatrix} \text{PE}_{(t+\Delta t, 2i)} \\ \text{PE}_{(t+\Delta t, 2i+1)} \end{pmatrix} = \begin{bmatrix} u & v \\ -v & u \end{bmatrix} \begin{pmatrix} \text{PE}_{(t, 2i)} \\ \text{PE}_{(t, 2i+1)} \end{pmatrix}$$

This implies that we can **compute** the **positional encoding** for a **shifted position** as a **rotation** of the "**original**" **positional encoding**.

 **IDEA BEHIND SINUSOIDAL POSITIONAL ENCODING**

The original authors **hypothesised** that this would **allow** the **model** to **more easily learn to attend by relative position** as **opposed** to learning **separate rules** for **each absolute position**.

In fact, let us look at how the **attention score** between a **query** i and a **key** j is computed:

$$\text{score}(i, j) = \mathbf{q}_i \cdot \mathbf{k}_j$$

Where:

$$\mathbf{q}_i = \mathbf{W}^Q(\mathbf{x}_i + \mathbf{p}_i)$$

$$\mathbf{k}_j = \mathbf{W}^K(\mathbf{x}_j + \mathbf{p}_j)$$

We thus obtain:

$$\text{score}(i, j) = \dots + (\mathbf{W}^Q \mathbf{p}_i) \cdot (\mathbf{W}^K \mathbf{p}_j)$$

NOTE: We have omitted the first three terms, because we are only interested in the last one.

🔍 OBSERVATION: POSITION-POSITION TERM

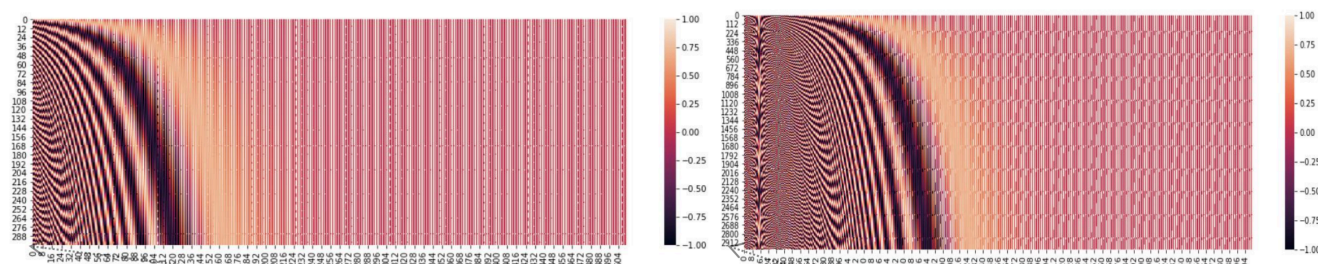
Part of the **attention score** between $\mathbf{q}_i$ and $\mathbf{k}_i$ depends on their **positional encodings** ($\mathbf{p}_i$ and $\mathbf{p}_j$).

Thus, it is reasonable to imagine that the fact that the **components** for each **frequency pair** can be expressed in terms of their **relative position** will **help the model** to rely on these, **instead of absolute positions**, thus reducing the "learning effort" (because the model learns how to handle a particular offset independently of its starting position).

Finally, we conclude this paragraph by addressing one more question: **why not let the model learn the positional encodings?**

The **boring answer** is that the original authors **tried this approach too**, but did **not notice significant differences in performance**, and thus opted for the idea above.

A **more interesting answer** can be understood by looking at the **heatmaps** below:



The heatmaps show the **positional encoding values** for a **sequence of 300 tokens** (left) and a **sequence of 3000 tokens** (right) (the position within the sequence is represented on the y axis, while the x axis represents the index of the embedding dimension).

Notice that the **positional encoding values** for the **"first" embedding dimensions** oscillate with **high frequencies** (observe a vertical band in the left part of the heatmaps), while the **values** for the **"late" embedding dimensions** oscillate with **much lower frequencies** (observe a vertical band in the right part of the heatmaps: colours change much less rapidly).

What is **more interesting** to us, though, is the following observation:

💡 SAME PATTERN FOR DIFFERENT LENGTHS

Even though the **two sequences** have **wildly different lengths**, the **pattern** with which the **values oscillate** is the **same in both cases**.

This suggests that **sinusoidal positional encoding** is **not influenced by sequence length**, which in turn means that it will be able to **handle in the same way** even **sequences of lengths** that were not seen during training.

ADD-AND-NORM AND LINEAR LAYERS

Each **layer** in the network (that is, both attention layers and feed-forward layers) is **followed** by an **add and norm layer**.

We can identify each layer as a pair of a **sublayer** and an **add and norm** function. Mathematically, the **output** of these **pairs** is defined as:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

Where $\text{Sublayer}(x)$ is the **function** implemented by the **attention/feed-forward layer**.

✍ NOTE: RESIDUAL CONNECTIONS

Notice that **adding the input** is simply due to the fact that **residual connections are employed** (see > [2.0 - CONVOLUTIONAL NEURAL NETWORKS > RESNET](#)). Furthermore, to facilitate these connections, **all sublayers** (in the original implementation) produce **outputs of dimension** $d_{\text{model}} = 512$.

As for the **layer normalisation** part, the idea behind it is very much **similar** to that of **batch normalisation** (see > [1.2 - DEEP NEURAL NETWORKS > BATCH NORMALISATION](#)), with the sole following difference:

✍ LAYER NORM VS BATCH NORM

- **LAYER NORM:** Mean and variance are computed across the **channels and spatial dimensions**.
- **BATCH NORM:** Mean and variance are computed across the **spatial and batch dimensions**.

Mathematically, this means **normalising** each **token** in the sequence as follows:

$$\hat{x}_{nk} = \frac{x_{nk} - \mu_n}{\sqrt{\sigma_n^2 + \epsilon}}$$

Where:

$$\mu_n = \frac{1}{K} \sum_{k=1}^K x_{nk} \quad \sigma_n^2 = \frac{1}{K} \sum_{k=1}^K (x_{nk} - \mu_n)^2$$

NOTE: $n = 1, \dots, T$ indexes the *tokens in the sequence*, k indexes the *features of each token*.

Furthermore, in addition to the attention sublayers, **each encoder and decoder block** contains a **fully connected feed-forward** network, which is **applied to each position in parallel** and with **shared weights** (that is, we apply the **same transformation to each token in parallel**).

More precisely, these feed-forward networks consist of **two linear transformations** with a **ReLU activation** function between them:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

The original implementation employed a *feed-forward network* with $d_{\text{model}} = 512$ **input neurons**, $d_{\text{ff}} = 2048$ **hidden neurons** and d_{model} **output neurons**.

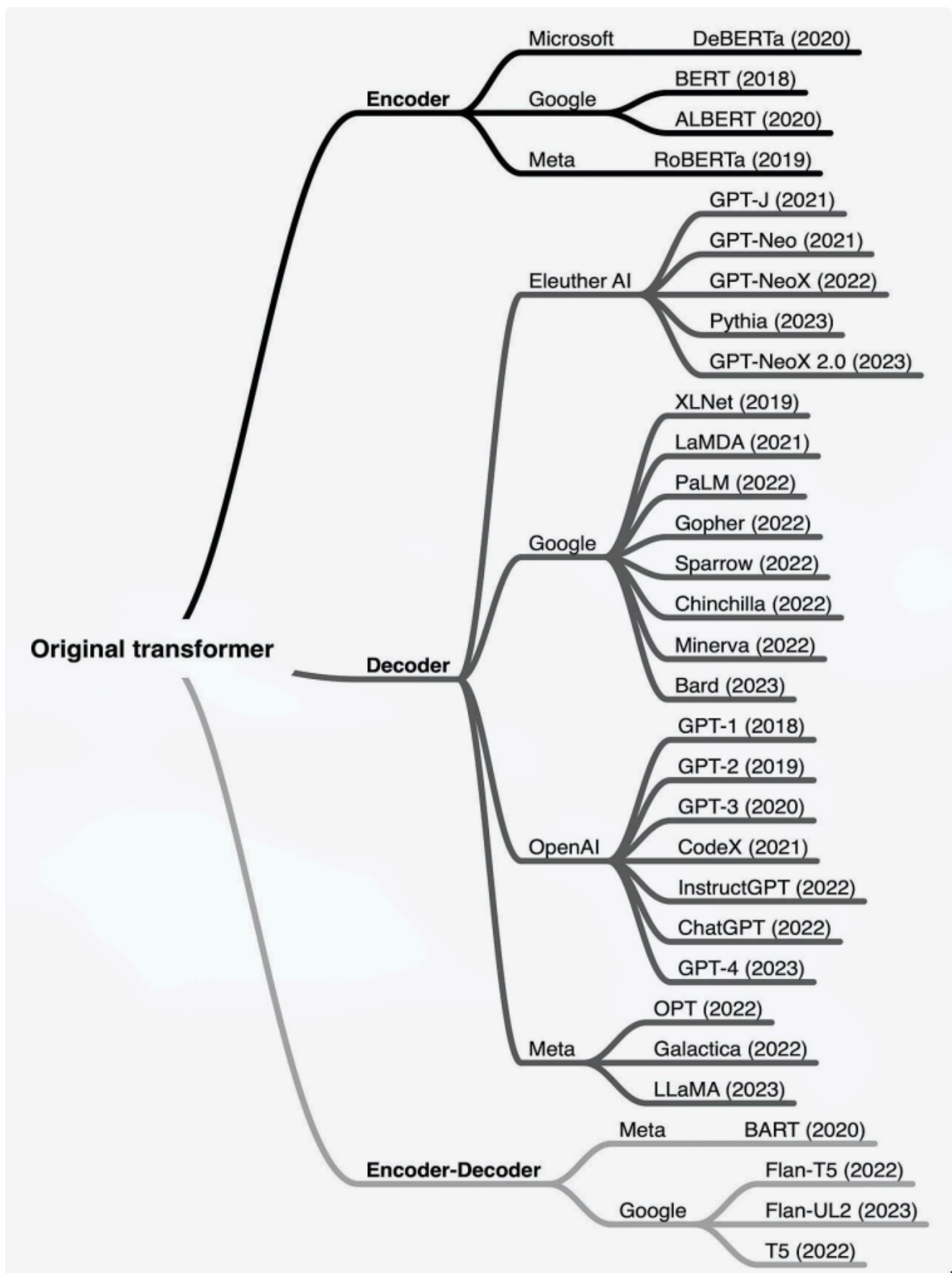
💡 INTUITION: LINEAR LAYER

By **briefly projecting** the **self-attention output** in a **higher-dimensional space**, we can ideally **capture further nuances** about the tokens, before feeding them to the next block.

ENCODER/DECODER-ONLY ARCHITECTURES

So far we have only discussed the "complete" **encoder-decoder architecture**, but there exist transformer models that **only employ either one or the other**.

Some examples are shown below:



In the complete **encoder-decoder** architecture, the **decoder** can generate **relevant output** thanks to the **rich representation** produced by the **encoder**.

💡 STRENGTH OF ENCODER-DECODER MODELS

This **deep interconnection** between **input representation** and **generated tokens** makes this kind of transformers **particularly good** at **sequence to sequence** tasks such as **language translation**.

As we can see from the figure above, though, **modern LLMs** (e.g. ChatGPT) are **decoder-only architectures**.

This is (mainly) due to the following reasons:

DECODER-ONLY ADVANTAGES

1. These models **can be trained** on massive amounts of **unlabeled text** to **simply predict the next token** based on the **previous ones**.
This **self-supervised** objective is a lot **easier to scale** compared to the **input-output datasets** required by **standard encoder-decoder** models (producing a dataset of *user query + target output* pairs would be *practically impossible*, while *simply feeding example text* is *much easier*).
2. By using a **single stack of decoders** we can significantly **reduce memory overhead** and **computational complexity** (one reason for this advantage is that we *do not need to keep encoder outputs in memory*, because we *do not compute cross-attention*).
3. It's been observed that at **large scales** (10B+ **parameters**) **decoder-only** models achieve **better zero-shot performance**.

All these reasons combined make **decoder-only architectures** inherently **better** at tasks such as **Q&A**, text **summarisation**, **coding** and, perhaps more importantly, **problem solving**.

DECODER-ONLY: PROBLEM SOLVING

Because these models learn to **predict the next token** with training processes that involve **enormous datasets**, they are incredibly **good** at **learning patterns**, even **subtle ones** that **humans likely can't see**.

This contributes to their **zero-shot capabilities**, which in turn are **fundamental for problem solving** tasks: even when the model is presented with a **problem it's never seen before**, it may **detect** some **obscure** (that is, to us humans) **pattern** that **links it to some training examples**, thus **producing** a somewhat **plausible answer**.

Furthermore, readers may remember that the first GPT (which stands for **generative pretrained transformer**) models available to the public used to produce some answers that were **plausible**, but **not quite correct**; while **current models** seem to produce

much better and **more pertinent answers**.

Indeed, these **improvements** are largely due to the **advantages** presented above.

Before discussing encoder architectures, it is worth stopping for a moment to discuss **pre-training**, a concept that is **very important** for modern LLMs.

PRE-TRAINING VS TRAINING

The "**training**" phase on a **huge unlabeled dataset** we mentioned above is actually what is called **pre-training**.

The "actual" **training phase** comes **after pre-training** and consists in **adapting the model** to a **more specific task** using a **smaller, labeled dataset** (for this reason, it is often called **fine-tuning**).

Intuitively, **pre-training** aims at **creating the "foundational intelligence"** of the model, while **training** aims at **specialising the model** for **domain-specific** performance.

Finally, **encoder-only models** also exist (e.g. **BERT**, Bidirectional Encoder Representations Transformer; and **ViT**, see > [ViT - VISION TRANSFORMER](#) below).

ENCODER-ONLY ARCHITECTURES

By only keeping the **encoder stack**, we may predict these models to be particularly good at **tasks** that require **rich contextual embeddings**.

Indeed, this is true: encoder-only models are especially good at tackling problems like **sentiment analysis**, **text classification**, or **named entity recognition**, where the model must comprehend the **full context** of a sequence.

TRANSFORMER EXAMPLES

Finally, we conclude this chapter by presenting **two example architectures**: **GPT-OSS**, an **open weight LLM** by OpenAI; and **ViT**, an **encoder-only architecture** that surpassed **CNNs** in **vision tasks**.

GPT-OSS

GPT-OSS (2025) is the **first open weight** model that OpenAI has **released since GPT-2** (2019), and it brought several key **architectural improvements** that enhanced **efficiency**, **scalability**, and overall **performance**.

NOTE: OPEN WEIGHT $\neq$ OPEN SOURCE

An **open weight** model usually comes with:

1. A **weight file** (e.g. .safetensors or .gguf format) which contains the **weights' values**.
2. A **config file** (config.json) which **describes** the **model architecture** (number of layers, number of attention heads, activation functions, position embedding type, hyperparameters, etc.).
3. The **runtime code**, which **instantiates** the **layer graph** and **loads the weights in memory**.

It **does not** come with **training code** and **training data** (which is why such models do **not** classify as **open source**).

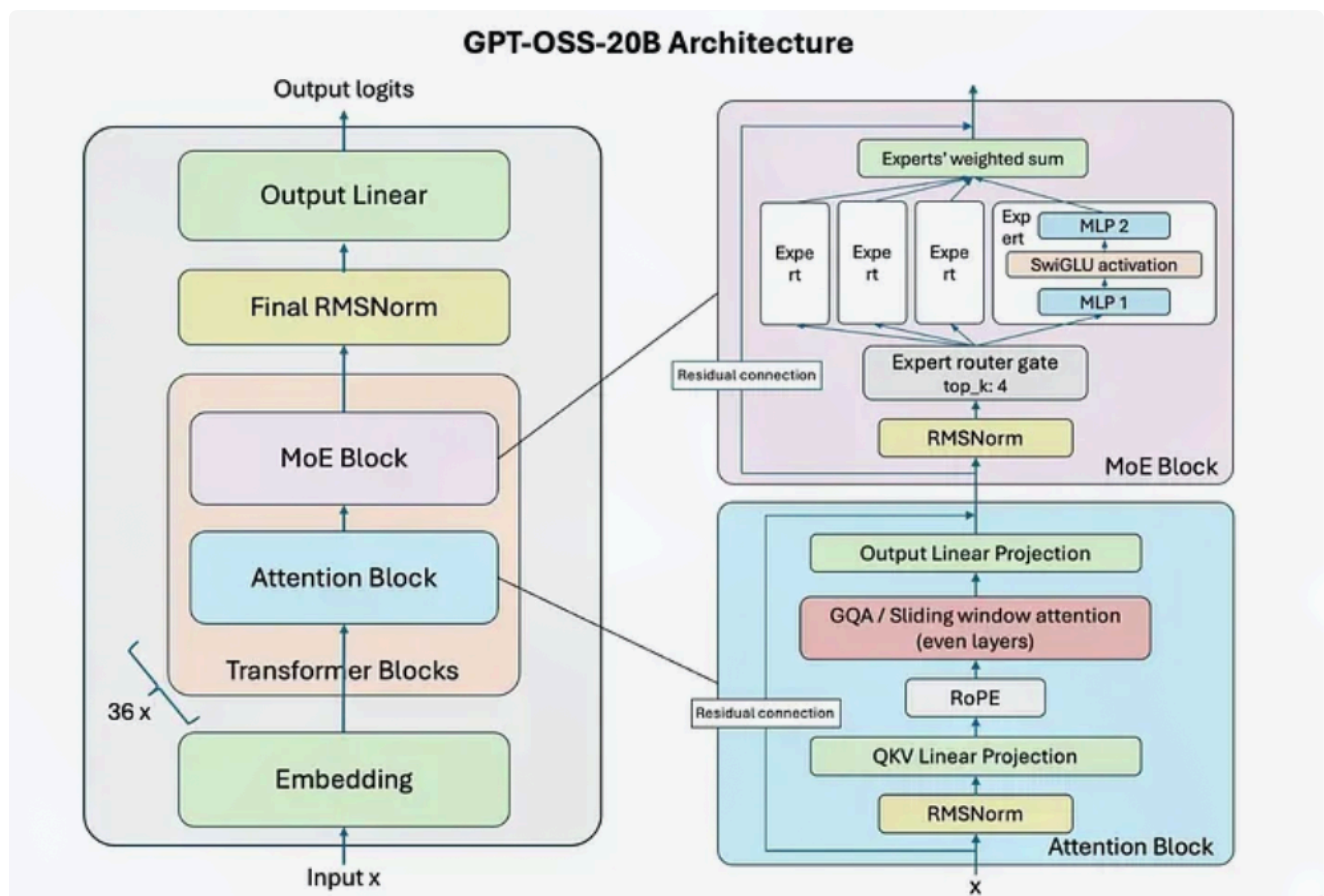
GPT-OSS is actually a **family of two models**:

1. **GPT-OSS-20B**
2. **GPT-OSS-120B**

The first is a **20 billion parameters model** designed for **broad accessibility**: it can be ran on **consumer GPUs** with at around 16GBs of VRAM and achieves **performance comparable to GPT-3.5**.

The second is a **120 billion parameters model** which requires **significantly** more **memory** and **computational power** (ideally, it requires GPUs with 80GB+ of VRAM, such as NVIDIA H100).

One of the **key features** of these models is that they are a **Mixture of Experts (MoE)**. To understand what this means, let us first look at their overall architecture (here, we are showing the 20B model variant, but the 120B has a similar architecture):



✍ MIXTURE OF EXPERTS

An **expert** is a **feed-forward network** (here, 2 MLPs with a SwiGLU activation function) within the main model.

During **inference**, only a **set number of experts** is **activated per token** (4 here), **chosen** by the **expert router gate**, which is **trained to select the best expert** (among those available) for a **particular token**.

Clearly, this implies a **huge reduction** in the **computational cost**, because **not all parameters are active** for **all tokens** (hence why the 120B model does not require six times as much memory as the 20B variant).

In any case, **MoE** is **not** the **only difference** with the standard transformer model we have discussed in the above paragraphs.

Namely, it uses **RoPE (Rotary Position Embedding)** instead of traditional positional encodings; **RMSNorm** instead of layer normalisation; and, lastly, a **sliding-window attention mechanism**.

Let us discuss the *last two* (see > [POSITIONAL ENCODING](#) for RoPE).

RMSNorm is employed to **reduce the expensive mean and variance computations** to **improve training efficiency** by using the **root mean square statistic**:

📖 RMSNORM

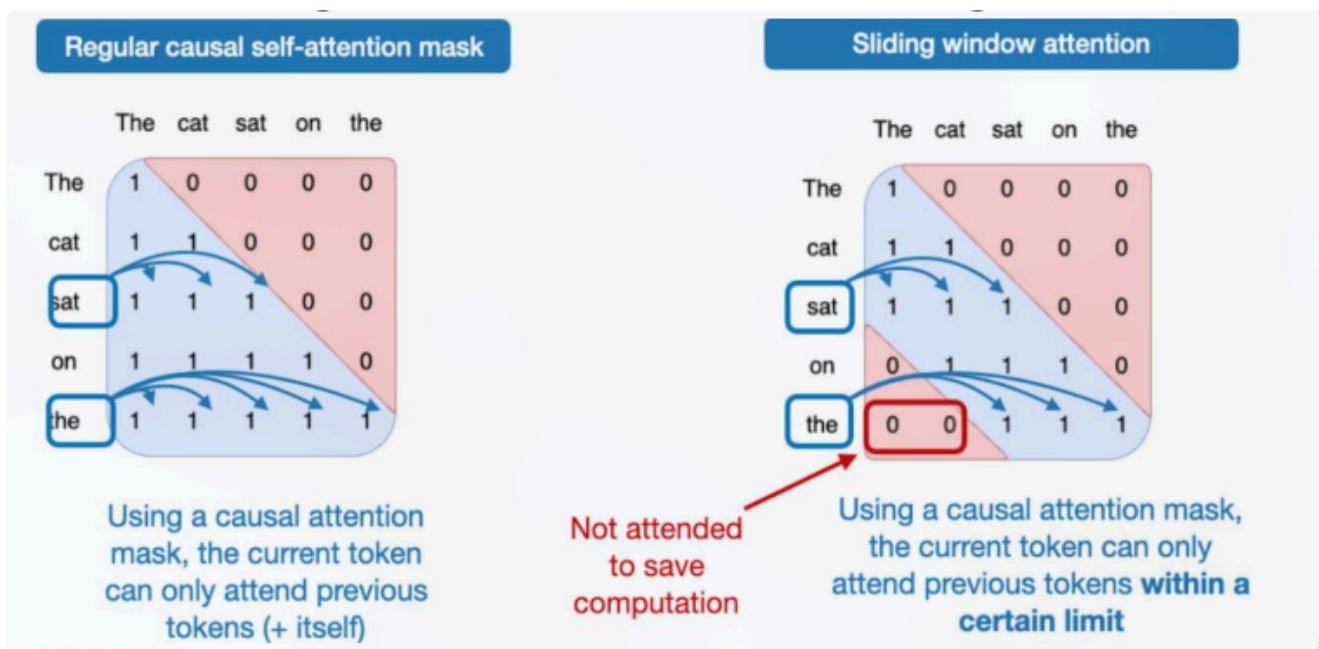
Each **token** x_i of the normalisation layer's input is **normalised** as follows:

$$\bar{x}_i = \frac{x_i}{\text{RMS}(\mathbf{x})} g_i, i = 1, \dots, n$$

Where g_i is the **gain parameter** (used to re-scale the standardised inputs, it is a *set of learnable parameters*) and:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}$$

Sliding-window attention computes **attention** on a **limited window** instead of the **whole sequence** to **save computations**, as shown below:



NOTE: GPT-OSS ATTENTION

GPT-OSS alternates between layers with **full-context attention** and layers that employ a **sliding-window attention** mechanism restricted to **128 tokens**.

ViT - VISION TRANSFORMER

ViT (*Vision Transformer*) is an **encoder-only** transformer **architecture** designed for **computer vision tasks**.

The model's **performance** was shown to be *acceptable* when trained on the *ImageNet* dataset (one million images), **great** when trained on **ImageNet-21k** (14M images), and **state-of-the-art** when pre-trained on **Google's internal JFT-300M** dataset (300M images).

One may wonder why **transformers outperform CNNs** on **large datasets**, but **fall behind** when applied to **smaller datasets**.

The answer lies in the **inductive bias** (set of assumptions that are intrinsic to the model) of the two architectures:

- **CNNs** have a **strong inductive bias** because their architecture intrinsically **assumes locality, translation invariance** (see > [2.0 - CONVOLUTIONAL NEURAL NETWORKS > CNNs AS TRANSLATION INVARIANT, CONSTRAINED MLPs](#)), and **feature hierarchy**.
- **Transformers**, on the other hand, have **weak inductive bias** because they **make no assumptions about the data**; instead, they rely on the **attention** mechanism to learn all the details.

PERFORMANCE AND INDUCTIVE BIAS

By making **fewer assumptions** about the **data** (*lower inductive bias*), ViT can **better adapt itself** to the **given task**.

This means that, when the **sample size** is **too small**, the model is **not able** to **learn the basic features**; it instead **overfits** due to the **large amount of parameters**.

Contrarily, when the **sample size** is **massive**, the **lack of assumptions** becomes an **advantage** that lets the model **adapt more freely** to the problem, unlike CNNs that *remain constrained*.

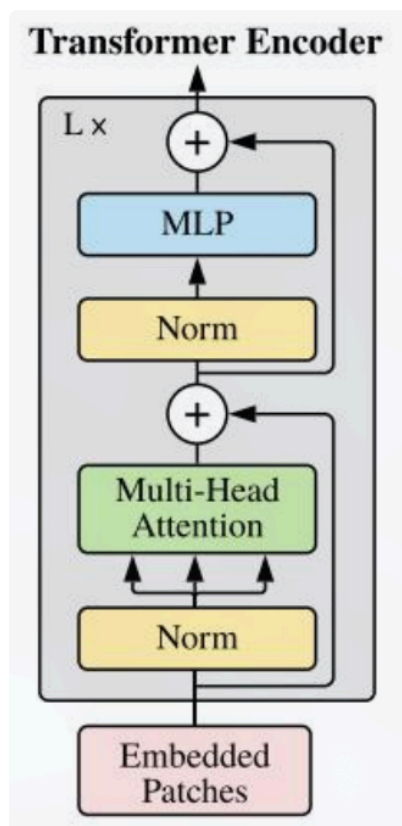
Particularly, we can train **vision transformers** as follows:

ViT TRAINING

1. **PRE-TRAINING** - Pre-train the model on a **large dataset** (e.g. JFT 300M) to let the **model learn features on its own**.
2. **FINE-TUNING** - Fine-tune (*train*) the model on a **specific sub-task** to let it **adapt to the specific details**.

Now, let us take a look at **ViT's architecture**.

As already stated, it is an **encoder-only model**, and it thus **stacks encoder blocks** like the one shown below:



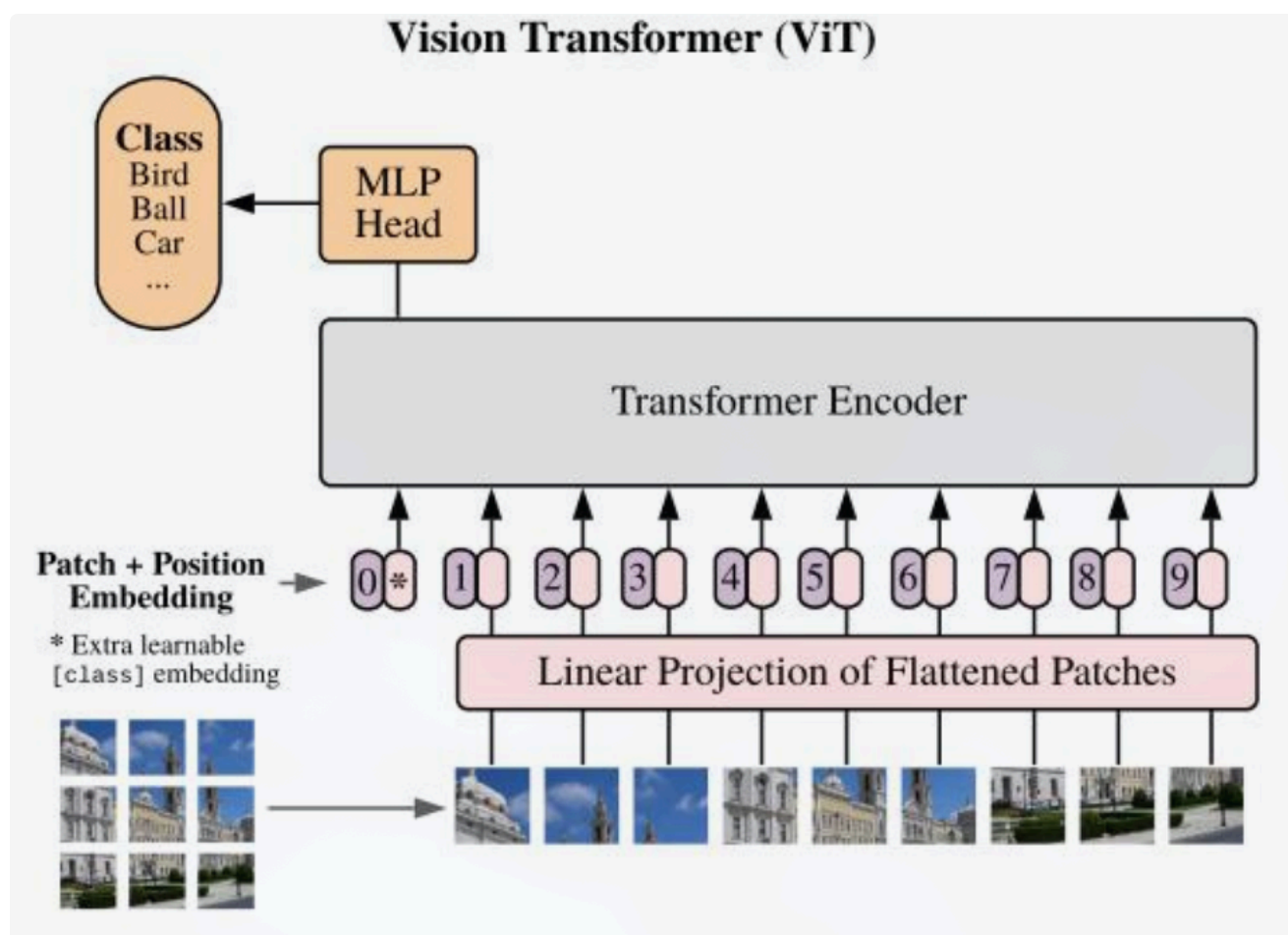
The **main problem** that the authors had to face was **figuring out how to feed images to**

a transformer.

The **proposed solution** was exceptionally **simple** and **clever**:

📄 FEEDING IMAGES TO A TRANSFORMER

1. **Split** the image into **patches**.
2. **Flatten** the patches.
3. Produce **lower-dimensional linear embeddings** from **each flattened patch**.
4. Add **positional embeddings**.



Notice that:

1. Each **patch** is **accompanied** by a **number** (label) that refers to the patch **position** in the original image.
2. In addition to the input patches, the input layer features an **additional class embedding** token. This is **randomly initialised** and its value is **learned during training** (it is not fed with the input).

To understand why the **extra class embedding** is **useful**, we first need to point out that the **embedding dimension** remains **fixed throughout the whole network**, so that **short residual connections** can be employed (that is, we simply carry the output to a further layer without the need to project it to another space).

Furthermore, we notice that the **final class prediction** is **only performed** on the basis of the **first token** of the **final encoder output**, and *not on the whole output sequence*.

💡 INTUITION: CLASS EMBEDDING

The **model learns** to use the **class embedding** token (and its equivalent in the successive layers) to **store context information** about the **entire image**.

At the end, we **only need** to pass this "**context token**" to the **MLP head** to **predict the final class**, because it will contain all the **relevant information** extrapolated from the processed patches.